

Software Development –Class & Objects

Python programming exercises

1. Complete the `Point` class we discussed in the lecture.
2. Add the following method to the `Point` class:

```
def isVertical(self,p):
```

where `p` is another `Point`. The method returns true if the given `Point` lines up vertically with the current `Point`, that is, if their x-coordinates are the same.

3. Add the following method to the `Point` class:

```
def slope(self, p):
```

where `p` is another `Point` and the method returns the slope of the line drawn between this `Point` and the given other `Point`. Use the formula $(y_2 - y_1) / (x_2 - x_1)$ to determine the slope between (x_1, y_1) and (x_2, y_2) .

4. Write a class called `Line` that represents a line segment between two `Points`. The following codes is the first version of the `Line` class

```
class Line:

    def __init__(self,newp1,newp2):

        self.p1=newp1

        self.p2=newp2
```

Then add the following methods to Your `Line` object:

```
def getP1(self):
```

Returns this `Line`'s first endpoint.

```
def getP2(self):
```

Return this `Line`'s second endpoint.

```
def getSlope(self):
```

Return the slope of this `Line`. The slope of a line between point (x_1, y_1) and (x_2, y_2) is equal to $(y_2 - y_1) / (x_2 - x_1)$.

```
def getLength(self):
```

Return the length of this `Line`.

Develop a client program to test your `Line` class.

5. The diagram below represents the design for the BankAccount class

Class: BankAccount
accountNumber:String accountName:String balance:float
__init__(String, String, float) getAccountNumber():String getAccountName():String getBalance():float deposit(float) withdraw(float): boolean

Detailed methods description:

__init__(String, String, float) is a constructor

getAccountNumber():String returns the account number

getAccountName():String returns the account name

getBalance():float returns the balance

deposit(float) accepts an item of type float and adds it to the balance

withdraw(float): Boolean accepts an item of type float and checks if there are sufficient funds to make a withdrawal. If there are not, then the method terminates and returns a value of false. If there are sufficient funds, however, the method subtracts the amount from the balance and returns a value of true.

Write the code for the BankAccount class.

Write client code to test out your BankAccount class; it should create two or three accounts, and use the methods of the BankAccount class to test whether they work according to the specification.

It is perfectly possible, and often very desirable, to create lists of objects.